Card Run Mines 20× — Math Pack for Certification

Oct 16, 2026 · @Bob

Card Run Mines 20× returns **93.0000% before rounding at every stop and every bomb count**, its DOUBLE OR NOTHING is exactly fair (100.0000%), and no bar policy can change the return. This is the math pack for the certification lab; every figure is exact (fractions) or checked against a simulation of the shipped game.

Summary

Build `card-run-mines-classic-launch.html`, sha256 `2f73b626f7889810d499dbb863632ff42645ff6c6300f3b8f3b47a0b24974426`, 152,362 bytes, run bar v170. RUN IT AGAIN and RUN HALF are not in this build. The base game, pays and RNG are unchanged since the earlier runs, and the whole pack was re-run on this exact build on 16 Oct 2026.

Item	Value
Base game RTP, every stop and bomb count	**93.0000%** before rounding to the cent
After rounding to the cent: $1 / $10 / $100	lowest stop 92.40% / 92.92% / 92.993%
Lowest anywhere on the $0.10 bet grid	**92.3636%** ($1.10, 1 live bomb, pick 5)
Typical play (4 live bombs, LOCK IN after 3 picks)	92.5217% at $1, 92.9843% at $10, 92.9959% at $100
Top picks, paid in full	23.25× (1 live), 27.90× (2), 25.4643× (3), 23.7667× (4)
DOUBLE OR NOTHING	C(19,6) / C(21,6) = 1/2 exactly, **100.0000%**
Whole round, any bar policy	Same as the base game (every bar action is a fair bet)
MAX WIN	20,000× the bet (an operator may lower it, never raise it)
Run cap	1,000× the build win; the double that crosses it pays in full

Item	Value
Bets	$1.00 to $100.00 in $0.10 steps
Rounding	Pay = floor(bet cents × multiplier), once, in integer cents

Two things the lab should know.

1. **Mines has no skill element that moves the return.** The player chooses a bomb count (1–4) and where to stop; each choice changes the volatility, not the RTP. Every stop returns 93% before rounding.
2. **Nothing exceeds 100% and nothing is ever clipped.** The only sub-game at 100% is the double, and it is exactly 1/2. A held amount plus its RUN BANK is a fair bet at every step, so every way of playing the bar returns what cashing out returns.

Rules as built

The player picks tiles on a 25-tile board; every safe pick raises the multiplier, a live bomb ends the round, and the player can LOCK IN at any point from 1.01×.

- **Board.** 25 tiles: 21 gems and 4 bomb tiles.
- **Bombs.** Before the first pick the player sets B = 1, 2, 3 or 4 bombs live. The other 4 − B bomb tiles are **shielded**: safe, and counted as a safe pick. Which tiles are live or shielded is part of the shuffle, fixed before the first pick.
- **Multiplier.** After k safe picks, M(k) = Fair(k) × 0.93, where Fair(k) = C(25,k) / C(25−B,k). It is held as an exact fraction.
- **Pay.** floor(bet cents × M(k)), rounded once, in integer cents. The board shows floor4(M) (two decimals).
- **LOCK IN** is offered once the display reaches 1.01×.
- **Top pick.** The first pick with M ⩾ 20 is the top pick. It pays in full and the board stops there. The "20×" in the name is a target, not a cap.
- **After any win** the run bar opens: CASH OUT, DOUBLE OR NOTHING or DOUBLE HALF, with a RUN BANK (see Run phase).

The DOUBLE OR NOTHING board. A separate 25-tile board of 19 gems, 2 live bombs and 4 duds, fixed before the first pick. The player picks tiles until 6 gems (win, the stake pays exactly 2×) or 1 live bomb (loss). A dud is neutral: it is not a pick, not a gem and gives no protection. The win chance is C(19,6) / C(21,6) = 1/2 exactly.

Pays and board math

Every pick on every ladder returns exactly 93% before rounding. The player reaches pick k with probability 1 / Fair(k), where Fair(k) = C(25,k) / C(25−B,k), and the pick pays M(k) = 0.93 × Fair(k), so the expected return of stopping at pick k is 0.93 exactly. The solver (`exact.py`) holds M(k) as a fraction, computes pays in the build's own integer cents, and a cross-check (`xcheck.js`) compares every ladder row with the build's `Classic.setMines` and `Classic.payC`.

How the cent is rounded. Pay = floor(bet cents × M(k)), once. The RTP column "at $1" is the exact return of that stop after rounding. It is below 93% only by the rounding, and the loss shrinks as the bet grows ($1 down to 92.40%; $10 down to 92.92%; $100 down to 92.993%).

Top picks (the first pick with M ≥ 20; it pays in full and the board stops there):

Live bombs	Top pick	Pays	Chance of reaching it
1	24	23.2500×	1 in 25.00
2	20	27.9000×	1 in 30.00
3	16	25.4643×	1 in 27.38
4	13	23.7667×	1 in 25.56

The ladders below show the board display, which is the multiplier cut (not rounded) at the fourth decimal, so the 3-bomb top pick reads 25.4642 and the 4-bomb top pick 23.7666. A pick below 1.01× (the 1-bomb first pick, 0.9687×) has no LOCK IN; the player must pick on.

The four ladders

1 live bomb — 24 picks

Pick	× bet	1 in	$1 pays	RTP at $1	$10 pays	RTP at $10	RTP at $100
1	0.9687	1.042	0.96	92.160%	9.68	92.928%	92.9952%
2	1.0108	1.087	1.01	92.920%	10.10	92.920%	92.9936%
3	1.0568	1.136	1.05	92.400%	10.56	92.928%	92.9984%
4	1.1071	1.190	1.10	92.400%	11.07	92.988%	92.9964%
5	1.1625	1.250	1.16	92.800%	11.62	92.960%	93.0000%

Pick	× bet	1 in	$1 pays	RTP at $1	$10 pays	RTP at $10	RTP at $100
6	1.2236	1.316	1.22	92.720%	12.23	92.948%	92.9936%
7	1.2916	1.389	1.29	92.880%	12.91	92.952%	92.9952%
8	1.3676	1.471	1.36	92.480%	13.67	92.956%	92.9968%
9	1.4531	1.562	1.45	92.800%	14.53	92.992%	92.9984%
10	1.5500	1.667	1.55	93.000%	15.50	93.000%	93.0000%
11	1.6607	1.786	1.66	92.960%	16.60	92.960%	92.9992%
12	1.7884	1.923	1.78	92.560%	17.88	92.976%	92.9968%
13	1.9375	2.083	1.93	92.640%	19.37	92.976%	93.0000%
14	2.1136	2.273	2.11	92.840%	21.13	92.972%	92.9984%
15	2.3250	2.500	2.32	92.800%	23.25	93.000%	93.0000%
16	2.5833	2.778	2.58	92.880%	25.83	92.988%	92.9988%
17	2.9062	3.125	2.90	92.800%	29.06	92.992%	92.9984%
18	3.3214	3.571	3.32	92.960%	33.21	92.988%	92.9992%
19	3.8750	4.167	3.87	92.880%	38.75	93.000%	93.0000%
20	4.6500	5.000	4.65	93.000%	46.50	93.000%	93.0000%
21	5.8125	6.250	5.81	92.960%	58.12	92.992%	93.0000%
22	7.7500	8.333	7.75	93.000%	77.50	93.000%	93.0000%
23	11.6250	12.500	11.62	92.960%	116.25	93.000%	93.0000%
24	23.2500	25.000	23.25	93.000%	232.50	93.000%	93.0000%

2 live bombs — 20 picks

Pick	× bet	1 in	$1 pays	RTP at $1	$10 pays	RTP at $10	RTP at $100
1	1.0108	1.087	1.01	92.920%	10.10	92.920%	92.9936%
2	1.1027	1.186	1.10	92.767%	11.02	92.935%	92.9944%
3	1.2077	1.299	1.20	92.400%	12.07	92.939%	92.9929%
4	1.3285	1.429	1.32	92.400%	13.28	92.960%	92.9950%
5	1.4684	1.579	1.46	92.467%	14.68	92.973%	92.9987%
6	1.6315	1.754	1.63	92.910%	16.31	92.967%	92.9955%
7	1.8235	1.961	1.82	92.820%	18.23	92.973%	92.9985%
8	2.0514	2.206	2.05	92.933%	20.51	92.979%	92.9968%

Pick	× bet	1 in	$1 pays	RTP at $1	$10 pays	RTP at $10	RTP at $100
9	2.3250	2.500	2.32	92.800%	23.25	93.000%	93.0000%
10	2.6571	2.857	2.65	92.750%	26.57	92.995%	92.9985%
11	3.0659	3.297	3.06	92.820%	30.65	92.972%	92.9990%
12	3.5769	3.846	3.57	92.820%	35.76	92.976%	92.9994%
13	4.2272	4.545	4.22	92.840%	42.27	92.994%	92.9984%
14	5.0727	5.455	5.07	92.950%	50.72	92.987%	92.9995%
15	6.2000	6.667	6.20	93.000%	62.00	93.000%	93.0000%
16	7.7500	8.333	7.75	93.000%	77.50	93.000%	93.0000%
17	9.9642	10.714	9.96	92.960%	99.64	92.997%	92.9992%
18	13.2857	14.286	13.28	92.960%	132.85	92.995%	92.9999%
19	18.6000	20.000	18.60	93.000%	186.00	93.000%	93.0000%
20	27.9000	30.000	27.90	93.000%	279.00	93.000%	93.0000%

3 live bombs — 16 picks

Pick	× bet	1 in	$1 pays	RTP at $1	$10 pays	RTP at $10	RTP at $100
1	1.0568	1.136	1.05	92.400%	10.56	92.928%	92.9984%
2	1.2077	1.299	1.20	92.400%	12.07	92.939%	92.9929%
3	1.3889	1.494	1.38	92.400%	13.88	92.936%	92.9959%
4	1.6082	1.729	1.60	92.522%	16.08	92.984%	92.9959%
5	1.8763	2.018	1.87	92.687%	18.76	92.984%	92.9992%
6	2.2074	2.374	2.20	92.687%	22.07	92.982%	92.9987%
7	2.6213	2.819	2.62	92.953%	26.21	92.989%	92.9992%
8	3.1455	3.382	3.14	92.835%	31.45	92.983%	92.9974%
9	3.8196	4.107	3.81	92.765%	38.19	92.984%	92.9990%
10	4.7010	5.055	4.70	92.978%	47.01	92.998%	92.9980%
11	5.8763	6.319	5.87	92.899%	58.76	92.994%	92.9988%
12	7.4790	8.042	7.47	92.888%	74.79	93.000%	92.9997%
13	9.7227	10.455	9.72	92.974%	97.22	92.993%	92.9997%
14	12.9636	13.939	12.96	92.974%	129.63	92.995%	92.9997%
15	17.8250	19.167	17.82	92.974%	178.25	93.000%	93.0000%

Pick	× bet	1 in	$1 pays	RTP at $1	$10 pays	RTP at $10	RTP at $100
16	25.4642	27.381	25.46	92.984%	254.64	92.999%	92.9997%

4 live bombs — 13 picks

Pick	× bet	1 in	$1 pays	RTP at $1	$10 pays	RTP at $10	RTP at $100
1	1.1071	1.190	1.10	92.400%	11.07	92.988%	92.9964%
2	1.3285	1.429	1.32	92.400%	13.28	92.960%	92.9950%
3	1.6082	1.729	1.60	92.522%	16.08	92.984%	92.9959%
4	1.9656	2.114	1.96	92.732%	19.65	92.969%	92.9970%
5	2.4281	2.611	2.42	92.687%	24.28	92.993%	92.9972%
6	3.0352	3.264	3.03	92.840%	30.35	92.993%	92.9995%
7	3.8446	4.134	3.84	92.889%	38.44	92.985%	92.9998%
8	4.9430	5.315	4.94	92.942%	49.43	92.999%	92.9987%
9	6.4640	6.951	6.46	92.942%	64.64	93.000%	92.9998%
10	8.6186	9.267	8.61	92.906%	86.18	92.993%	92.9991%
11	11.7527	12.637	11.75	92.978%	117.52	92.994%	92.9996%
12	16.4538	17.692	16.45	92.978%	164.53	92.995%	92.9997%
13	23.7666	25.556	23.76	92.974%	237.66	92.997%	92.9997%

Player choices and the stopping rule

The player's choices change how much the return varies, not what it is. All tiles are fixed before the first pick and no information is revealed that could improve a pick, so the only decisions are the number of live bombs (1–4) and the pick at which to stop. Every stop on every ladder returns 93% before rounding, so there is no best or worst strategy on the board.

Base game by strategy (exact):

Strategy	RTP at $1	RTP at $10	RTP at $100	SD at $10 (× bet)	Hit rate	Top pay at $10	Top pay, 1 in
Best at $10 (13 stops tie; shown: 3 live, LOCK IN at pick 15)	92.9739%	93.0000%	93.0000%	3.9639	5.217%	17.825×	19.167

Strategy	RTP at $1	RTP at $10	RTP at $100	SD at $10 (× bet)	Hit rate	Top pay at $10	Top pay, 1 in
Typical (4 live, LOCK IN after 3 picks)	92.5217%	92.9843%	92.9959%	0.7941	57.826%	1.608×	1.729
Play to the top pick, 1 live bomb	93.0000%	93.0000%	93.0000%	4.5561	4.000%	23.250×	25.000
Play to the top pick, 2 live bombs	93.0000%	93.0000%	93.0000%	5.0082	3.333%	27.900×	30.000
Play to the top pick, 3 live bombs	92.9843%	92.9990%	92.9997%	4.7766	3.652%	25.464×	27.381
Play to the top pick, 4 live bombs	92.9739%	92.9974%	92.9997%	4.6084	3.913%	23.766×	25.556
Random player (1–4 bombs, stop uniform up to the top pick)	92.7975%	92.9812%	92.9982%	2.1027	38.006%	27.900×	2,400

Where a row is below 93.0000% the whole difference is the rounding of the pay to the cent. The lowest stop on the whole $0.10 grid is 92.3636% ($1.10, 1 live bomb, pick 5), above the 75% (GLI-19 §4.7) and 80% (Missouri 11 CSR 45-5.190) minimums.

Disclosure. The rules sheet shows the pays and the chance of reaching each pick for the chosen bomb count, states that LOCK IN timing and bomb count do not change the return, and gives the return table: build, any stop and any live bombs 93.0%; lowest at small bets (rounding) 92.3%; DOUBLE OR NOTHING 100.0%.

The stop rule holds at every bet. The board stops at the top pick, and not before, on all 991 bets of the $0.10 grid with 1–4 bombs (0 failures) and on every cent stake from $0.02 to $10,000.00 in the solver (0 failures). Through the build's own `Classic.payC` it was checked on every stake to $1,000.00 (399,996 bet and bomb pairs, 0 failures). An earlier build worked the pay out in floating point and missed the 20× stop on 79 of the 991 bets; pays are now integer cents.

Run phase: DOUBLE OR NOTHING and the run bar

DOUBLE OR NOTHING wins with probability exactly 1/2, so it returns exactly 100.0000% of the amount run. The player must find 6 gems on a board of 19 gems, 2 live bombs and 4 duds before hitting a live bomb. Duds are neutral. The chance is C(19,6) / C(21,6) = 1/2 (the duds drop out of the count); there are no ties, and a win pays exactly 2×.

The bar

After any win, including a return of the bet or less, the bar offers **CASH OUT**, **DOUBLE OR NOTHING** and **DOUBLE HALF**, with a **RUN BANK** escrow. The bar is the shared module `run-bar.js` v170, embedded byte for byte in the build.

- CASH OUT pays the RUN BANK plus the held amount.
- DOUBLE OR NOTHING plays the double board with all of the held amount.
- DOUBLE HALF stakes half the held amount (rounded down to the cent) and puts the rest, with any odd cent, in the RUN BANK.
- A won double makes the held amount 2× the stake; the RUN BANK stays. A lost double returns the RUN BANK as the held amount (the banked-bust rule); with an empty RUN BANK the round ends at 0.
- After every result, in cents: clip the held amount to MAX WIN minus RUN BANK; end at MAX WIN; end at the **1,000× run cap** (1,000 × the build win; the double that crosses it pays in full); otherwise show the bar.
- DOUBLE needs a held amount of at least $0.02 and RUN BANK + 2 × held within MAX WIN. DOUBLE HALF needs half of at least $0.02 (held of $0.04 or more) and RUN BANK + the kept half + 2 × half within MAX WIN. A bar with neither shows CASH OUT alone.
- Doubles are exact whole-cent doublings: no rounding at small amounts. RUN IT AGAIN and RUN HALF are not in this build.

Checked against the build's own code. `xcheck.js` runs `RunBar.compute` (v170) on 3,078 states (2,420 bars, 229 at MAX WIN, 429 at the run cap) and compares every field — mode, held, banked, half, keep, double, doubleHalf, halfPill, cashOutOnly, nMax — with the solver's rules: all agree, and the build has no RUN IT AGAIN or RUN HALF key.

Why the bar is fair. Every action is a fair bet on the money at stake, and DOUBLE or DOUBLE HALF is shown only when the doubled amount cannot pass MAX WIN, so nothing is ever clipped. The next section turns that into the exact round-level result.

Round-level RTP: every bar policy returns the base RTP

However a player uses the bar, a round returns exactly what cashing out returns: 92.5217% at $1, 92.9843% at $10 and 92.9959% at $100 for the typical play (4 live bombs, LOCK IN after 3 picks). Each double is exactly fair, so the money at stake plus the RUN BANK is a martingale and no policy can raise or lower its mean.

The argument

Held amount plus RUN BANK changes only through DOUBLE and DOUBLE HALF. Each is a bet that wins with probability exactly 1/2 and pays exactly 2×, with no tie, no rounding (whole-cent doubling) and no clipping (a double is shown only when it cannot pass MAX WIN). The total therefore has an expected change of zero at every step. It is bounded by MAX WIN and the round ends with probability 1, so by optional stopping the expected final payout of any policy equals the expected payout of cashing out at once. That includes the best possible policy: there is no worst case to find above the base RTP.

Named policies (typical base play, exact)

Policies play 4 live bombs and LOCK IN after 3 picks, then act at the bar.

Policy	Rule	RTP at $1 / $10 / $100	SD at $10	Hit rate	Top pay at $10
P0	Cash out	92.5217% / 92.9843% / 92.9959%	0.7941	57.826%	$16.08
P2	DOUBLE OR NOTHING once, then cash out	same	1.4580	28.913%	$32.16
P3	DOUBLE while shown, to the cap	same	39.1179	0.056%	$16,465.92
P4	A random shown button at every bar	same (derived mean)	simulation only	about 41.2%	simulated max $4,534.56
P5	DOUBLE HALF once, then cash out	same	1.0022	57.826%	$24.12

(P1, RUN IT AGAIN once, was retired with that mechanic.) Hit rate is the share of rounds that return more than 0; a lost half still returns the RUN BANK, so P5 has P0's hit rate.

P5 closed form. A base pay of c cents held at the first bar becomes c + ⌊c/2⌋ with probability 1/2 (the half doubled, plus the banked half) and ⌈c/2⌉ with probability 1/2 (the half lost, the RUN BANK returned), whenever ⌊c/2⌋ ⩾ 2 cents. The mean is exactly c. `exact.py` asserts this equals the full enumeration, fraction for fraction. At $1 the base pays $1.60, so P5 pays $0 (97/230), $0.80 (133/460) or $2.40 (133/460).

P4. The state graph cycles through the RUN BANK, so the distribution has no closed form and is simulated; its mean is the derived value above.

Rounding, limits and RNG

The only rounding is one floor on the base pay, and the doubles add none. At $1 it costs up to 0.6 points on a stop; at $100 it costs under 0.01.

- **Rounding.** Pay = floor(bet cents × M(k)) as a BigInt, once. The lowest stop after rounding is 92.40% at $1, 92.92% at $10 and 92.993% at $100; the lowest anywhere on the $0.10 grid is 92.3636% ($1.10, 1 live bomb, pick 5). The rules sheet shows this as "lowest at small bets 92.3%". Doubles are exact whole-cent doublings, and DOUBLE HALF stakes the floor of half the cents and banks the rest.
- **Bets.** $1.00 to $100.00 in $0.10 steps; typed amounts snap down to the grid and are clamped.
- **MAX WIN.** 20,000× the bet; an operator may set it lower, never higher. Every payout is clipped to it, and a double that could pass it is not offered.
- **Run cap.** 1,000× the build win (the first win); the double that crosses it pays in full, so a run can pay more than 1,000×.
- **Minimums.** $0.02 for DOUBLE OR NOTHING and $0.04 for DOUBLE HALF (a half of at least $0.02).
- **RNG.** Every outcome — the tile shuffle and which bomb tiles are shielded or live — comes from a Fisher–Yates shuffle on `crypto.getRandomValues` with rejection sampling (`rngInt` , exactly uniform, no modulo bias). The game throws if `crypto` is missing; there is no `Math.random` fallback. `Math.random` is used only to tilt the tile-deal animation. Production replaces `rngU32` with the platform RNG (open item).

Simulation evidence

All 15 exact values sit inside their simulation's 95% interval, so no follow-up seeds were needed. The simulations run the build's own scripts in Node, playing only through the game's handlers (lobby bet field, PLAY, bomb selector, tiles, LOCK IN, the run bar, PLAY AGAIN). Every outcome comes from the build's crypto RNG, replaced by a seeded xoshiro128** stream.

Run	Rounds	Simulated RTP [95% CI]	Exact	z
Base mix, $10 (bombs 1–4, LOCK IN after 1–6 picks)	2,000,000	93.0338% [92.9361, 93.1315]	92.9583%	+1.51

Run	Rounds	Simulated RTP [95% CI]	Exact	z
Play to the top pick, $10	1,000,000	93.8011% [92.8680, 94.7341]	92.9991%	+1.68
Play to the top pick, $1.10	1,000,000	92.8438% [91.9155, 93.7721]	92.9919%	−0.31
P0 cash out, $10	200,000	93.1458% [92.7979, 93.4937]	92.9843%	+0.91
P0 cash out, $100	100,000	92.8398% [92.3474, 93.3322]	92.9959%	−0.62
P2 double once, $10	200,000	93.2351% [92.5955, 93.8746]	92.9843%	+0.77
P2 double once, $100	100,000	93.7291% [92.8232, 94.6350]	92.9959%	+1.59
P3 double to the cap, $10	200,000	93.0324% [75.8838, 110.1811]	92.9843%	+0.01
P3 double to the cap, $100	100,000	97.1610% [72.3756, 121.9464]	92.9959%	+0.33
P4 random button, $1	200,000	92.5173% [91.5538, 93.4808]	92.5217%	−0.01
P4 random button, $10	200,000	93.6734% [92.4736, 94.8732]	92.9843%	+1.13
P4 random button, $100	100,000	92.1074% [90.8816, 93.3332]	92.9959%	−1.42
P5 DOUBLE HALF once, $1	200,000	92.2244% [91.7874, 92.6614]	92.5217%	−1.33
P5 DOUBLE HALF once, $10	200,000	92.8089% [92.3698, 93.2481]	92.9843%	−0.78
P5 DOUBLE HALF once, $100	100,000	92.9869% [92.3654, 93.6085]	92.9959%	−0.03

The base mix is the exact average over bomb counts 1–4 and LOCK IN after 1–6 picks (rounds alternate through them); the exact value 92.9583% is that same mix. P3 is too heavy-tailed to test its mean tightly (hit rate 0.056%: about 110 winning rounds at $10, 59 at $100); its exact value rests on the enumeration. P4′s expected value is the derived one in the round-level section.

Pay distributions. For P0, P2 and P5 every pay was compared with the exact distribution (chi-square): P0 $10 0.827 and $100 0.387 (1 df); P2 $10 0.591 and $100 2.528 (1 df); P5 $1 2.982 (p 0.225), $10 0.908 (p 0.635) and $100 0.317 (p 0.853) (2 df). No pay outside the exact support was ever seen.

Small amounts. The $1 runs go through the shipped code with P4 and P5: the LOCK IN pays $1.60, so halves and RUN BANK returns go down to $0.02. P4 at $1 ended 12,396 rounds on a pay of $0.03–$0.99, the smallest $0.03, and every round ended by CASH OUT or a bust. DOUBLE HALF was shown at every one of the 115,271 P5 bars at $1; P5 paid only $0, $0.80 and $2.40, as the closed form requires.

Cross-check of the solver against the build. `xcheck.js` (7 checks, all pass): the exact ladders equal the build′s `Classic.setMines` display and top pick for 1–4 bombs; every pay at $1, $10 and $100 equals `Classic.payC`; the stop rule on 399,996 bet and bomb pairs; `RunBar.compute` on 3,078 states; no RUN IT AGAIN code remains.

Same seeds as the previous run. 66 chunks exist in both runs; 58 are identical round for round (all 20 base chunks, all 20 top-pick chunks, and P0, P2, P3 at $10 and $100), so the base game is bit-identical. The 8 that differ are all P4 (its random choices changed with the bar). New chunks: P5 at $1, $10 and $100.

Reproduce, files and open items

The pack folder `mines/` reproduces every number here with Python 3 (standard library only) and Node (no packages). `exact.py` parses every constant and rule fragment from the build text and stops if one differs; it also asserts that no RUN IT AGAIN or RUN HALF code remains. Every output records the build′s sha256.

```
cd mines
python3 -I exact.py > out/exact.txt          # exact ladders, strategies,
policies, run-bar vectors
node -r ./seedcrypto.js xcheck.js > out/xcheck.log   # solver v the build's
own code (7 checks)
./run_sims.sh 1                              # 74 simulation chunks, seeds in
README.txt (about 1 h 30 min)
python3 -I summarize.py > out/summary.txt    # CI checks, chi-square,
results.json, par.txt
```

File	What it does
`exact.py`	Exact math in fractions: ladders, every stop at $1/$10/$100, top picks, strategies, the whole-grid scan, the stop rule, the double, policies P0, P2, P3, P5 by full enumeration, P4's derived mean, 3,078 run-bar test states
`xcheck.js`	Cross-checks the solver against the build's own `Classic.setMines`, `Classic.payC` and `RunBar.compute`
`sim.js`, `harness.js`, `seedcrypto.js`	Play the shipped code through its own handlers with a seeded RNG
`run_sims.sh`, `summarize.py`	Run every chunk; merge, compare with the exact value (95% CI), chi-square
`par.txt`, `results.json`, `README.txt`	PAR tables, every headline number as data, every command and seed

Open items for the lab and for the team.

1. **Nothing in the maths needs a decision.** Mines has no rule change in this submission; the return is 93% at every stop and every round policy.
2. **The rules sheet says "lowest at small bets 92.3%".** The exact lowest on the grid is 92.3636%, which rounds to 92.4%; the sheet figure is cut down, so it is on the cautious side. Say if you want 92.4%.
3. **A display bug, no effect on any pay.** The NEXT strip shows "×NaN" for the cells past the top pick when 2, 3 or 4 live bombs are chosen and the player is within 3 picks of the top pick. The one-line fix is `if (k > G.top) break;` in the NEXT loop. It is not applied.

4. **RNG.** Production replaces `rngU32` with the platform RNG and decides outcomes server-side; that is certified separately.
5. **The §12 tests (1–17)** from the original hand-off notes need definitions, or to be declared out of scope.